UAV Path Planning

08/16/2026 16:24:29
https://chatgpt.com/c/6a804056-978c-83e8-83f9-b0f9709b87d7

ba5be526-46d7-43db-9755-56877eaf09de.pdf hey first explain me this...

I've gone through the paper. This is a UAV path-planning paper that...

what is the difference between knots and control points

This is the most important concept in B-Splines. Many people initially...

0602545d-34e1-4c8c-ade0-54f3b41e553b.pdf now similarly tell me abou...

Thinking ● Accessed uploaded files and read the second PDF’s opening...

cc1e367f-634a-472e-8b27-657b323499fd.md what my faculty was to...

Thinking ● Evaluating the repository I’m checking the documented...

Pasted markdown(20260815-143639).md ok, these are very great but fir...

Yes. That is the right strategy. Don't jump into extensions yet. First make ...

ba5be526-46d7-43db-9755-56...

hey first explain me this paper in detail.

8/15/2026, 4:02:53 PM

I've gone through the paper. This is a UAV path-planning paper that combines **B-Splines + Genetic Algorithms (GA)** to generate smoother and safer drone trajectories. `ba5be526-46d7-43db-9755-56877eaf09de.pdf`

Since you're learning graphs, algorithms, optimization, and are interested in drones, I'll explain it as if you're preparing for an interview or research discussion.

1. What problem is the paper solving?

Suppose a drone uses A* or RRT to find a path.

The output may look like:

```
Start
  |
  |
  *
   \
    *
      \
       *
         \
          Goal
```

The path consists of many straight segments.

Problems:

- Sharp turns
- High acceleration requirements
- Energy wastage

- Hard for real UAVs to follow smoothly

The paper focuses on the **path smoothing stage** after the initial path is found. `ba5be526-46d7-43db-9755-56877eaf09de.pdf`

---

2. Existing pipeline

Typical UAV navigation:

Step 1: Global planner

Examples:

- A*
- D*
- RRT
- PRM

Output:

```
Waypoint1
Waypoint2
Waypoint3
Waypoint4
```

Step 2: Trajectory smoothing

Convert those waypoints into a smooth curve.

This paper improves Step 2. `ba5be526-46d7-43db-9755-56877eaf09de.pdf`

---

3. Why B-Splines?

The core idea is:

Instead of connecting waypoints using straight lines,

fit a smooth mathematical curve.

The chosen curve:

Cubic B-Spline

A B-Spline is defined by:

- Control points
- Knot vector
- Basis functions

Mathematically:

$$C(u) = \sum d_i N_{i,k}(u)$$

where:

- d_i = control points
- $N_{i,k}$ = spline basis functions
- u = parameter along curve

`ba5be526-46d7-43db-9755-56877eaf09de.pdf`

---

4. Why not just use a polynomial?

Polynomial fitting often suffers from:

- Oscillations
- Poor local control

If you move one point:

Entire curve changes.

B-Splines have:

Local Control

Move one control point:

Only nearby region changes.

Convex Hull Property

Curve stays within the region formed by control points.

Useful for obstacle avoidance.

C² Continuity

Position, velocity, and acceleration change smoothly.

Perfect for drones. `ba5be526-46d7-43db-9755-56877eaf09de.pdf` `ba5be526-46d7-43db-9755-56877eaf09de.pdf`

---

5. What is the actual research contribution?

The authors say:

Regular B-Splines have a problem.

They require selecting:

Knot Vector

Think of knots as:

"Where the spline is allowed to change behavior."

Bad knot placement:

Too few knots

```
Curve too smooth
Misses sharp turns
```

Too many knots

```
Overfitting
High computation
```

The paper's key idea:

> *Automatically optimize knot locations using a Genetic Algorithm.* `ba5be526-46d7-43db-9755-56877eaf09de.pdf`

---

6. Optimization formulation

The paper converts smoothing into an optimization problem.

Inputs:

```
Raw waypoints
```

Outputs:

```
Best B-Spline
```

Requirements:

1. Stay close to original path

Data fidelity

2. Low curvature

Smooth turns

3. Respect UAV dynamics

Velocity limits

Acceleration limits

4. Short path

Energy efficient

5. Reasonable knot distribution

Avoid overfitting

`ba5be526-46d7-43db-9755-56877eaf09de.pdf`

---

7. Objective Function

The optimization target is:

$$J = w_1 F_{data} + w_2 F_{curv} + w_3 F_{dyn} + w_4 F_{len} + w_5 F_{knot}$$

Meaning:

Fdata

Stay close to original waypoints

Fcurv

Penalize sharp turns

Fdyn

Respect velocity and acceleration

Flen

Minimize total distance

Fknot

Prevent bad knot placement

`ba5be526-46d7-43db-9755-56877eaf09de.pdf`

This is the heart of the paper.

---

8. Curvature Constraint

Curvature measures how sharply the drone turns.

Example:

Low curvature

```
---------
```

Easy flight.

High curvature

```
)
```

Hard flight.

The paper enforces:

$$\kappa \leq \kappa_{max}$$

so the drone never attempts impossible turns. `ba5be526-46d7-43db-9755-56877eaf09de.pdf`

---

9. Why Genetic Algorithm?

The knot-selection problem is highly non-linear.

Gradient methods can get trapped.

GA explores globally.

Each chromosome = a knot vector.

Example:

```
[0,0,0,0.2,0.5,0.8,1,1,1]
```

A chromosome represents one candidate knot arrangement. `ba5be526-46d7-43db-9755-56877eaf09de.pdf`

---

10. GA Operations

Selection

Keep better knot vectors.

Crossover

Mix two good knot vectors.

```
Parent1
0.1 0.3 0.5 0.8

Parent2
0.2 0.4 0.6 0.9
```

```
Child
0.1 0.3 0.6 0.9
```

Mutation

Randomly adjust knot positions.

This helps escape local minima. `ba5be526-46d7-43db-9755-56877eaf09de.pdf`

---

11. Second Innovation

After GA optimizes knots,

the paper further adjusts control points.

Why?

Even with good knots:

```
Curve may still turn too sharply
```

So they perform:

Curvature-aware refinement

Control points are moved slightly to reduce curvature violations.

This ensures:

- smoother turns
- lower acceleration
- safer flight

`ba5be526-46d7-43db-9755-56877eaf09de.pdf`

---

12. Full Algorithm

The paper's workflow:

```
Raw Waypoints
      |
      v
```

```
Initial B-Spline
      |
      v
Genetic Algorithm
(optimize knots)
      |
      v
Compute Best Spline
      |
      v
Curvature Refinement
(adjust control points)
      |
      v
Final Smooth Path
```

`ba5be526-46d7-43db-9755-56877eaf09de.pdf`

---

13. Simulation Setup

Environment:

```
2000m x 2000m x 500m
```

Obstacles:

- Buildings
- Towers
- Cuboids
- Cylinders

Start:

```
(0,0,20)
```

Goal:

```
(2000,1800,200)
```

`ba5be526-46d7-43db-9755-56877eaf09de.pdf` `ba5be526-46d7-43db-9755-56877eaf09de.pdf`

14. Compared Methods

The authors compare:

Method 1

Raw polyline

(no smoothing)

Method 2

Uniform B-Spline

Method 3

Classical B-Spline

(chord-length knots)

Method 4

Enhanced Adaptive B-Spline

(their method)

`ba5be526-46d7-43db-9755-56877eaf09de.pdf`

15. Results

Method	Path Length	Curvature
Raw	2984.6	0.65
Uniform	2840.8	0.56
Classical	2803.2	0.49
Proposed	2731.4	0.38

`ba5be526-46d7-43db-9755-56877eaf09de.pdf`

16. Why is this important?

The proposed method achieved:

Shortest Path

2731.4 m

Lowest Curvature

0.38

Best Straightness Ratio

1.013

Meaning:

- less turning
- less energy consumption
- smoother motion
- safer UAV operation

`ba5be526-46d7-43db-9755-56877eaf09de.pdf`

---

17. Limitations

The paper is not perfect.

Authors admit:

Dynamic obstacles not tested

Mostly static environment.

GA is computationally expensive

Real drones may struggle.

Real-world wind effects ignored

Depends on upstream planner

If A*/RRT path is poor, smoothing can't fully fix it.

`ba5be526-46d7-43db-9755-56877eaf09de.pdf`

---

18. Research-level Critique

If I were reviewing this paper:

Strengths

✅ Clear mathematical formulation

✅ Good use of B-Splines

✅ GA-based adaptive knot optimization is reasonable

✅ Curvature constraints are practical

✅ Better results than classical B-Splines

Weaknesses

❌ Only simulation study

❌ No real UAV experiments

❌ Only 5 runs

❌ No statistical significance testing

❌ Doesn't compare against modern methods like:

- MPC trajectory generation
- Minimum snap trajectories
- Kinodynamic planning
- Reinforcement Learning planners

❌ Improvement is incremental rather than revolutionary

Interview / Viva One-Line Summary

> *The paper proposes a two-stage UAV path smoothing framework where a Genetic Algorithm optimizes B-Spline knot placement and a curvature-aware refinement step*

> *adjusts control points, producing shorter, smoother, and dynamically feasible UAV trajectories than conventional B-Spline methods.* `ba5be526-46d7-43db-9755-56877eaf09de.pdf`

Since you're interested in drones, the next useful step would be understanding **B-Splines from scratch (control points, knot vectors, basis functions, De Boor algorithm, and why C² continuity matters)**. That's the mathematical foundation behind almost the entire paper.

8/15/2026, 4:02:55 PM

what is the difference between knots and control points

8/15/2026, 5:17:07 PM

This is the **most important concept** in B-Splines. Many people initially confuse knots and control points because both affect the curve, but they play completely different roles.

Think of a B-Spline as:

```
Control Points  --> Shape of curve
Knots           --> How the curve uses those control points
```

---

1. Control Points

Control points are actual points in space.

Example:

```
P0 = (0,0)
P1 = (2,4)
P2 = (5,3)
P3 = (8,7)
P4 = (10,2)
```

These form the control polygon:

```
      P3
       *
      /
 P1 *
    / \
P0*   *P2
         \
          *P4
```

The spline curve bends according to these points.

What happens if I move a control point?

Suppose:

```
P2 = (5,3)
```

becomes

```
P2 = (5,8)
```

Then the nearby part of the curve moves upward.

So:

> *Control points determine the geometry (shape) of the curve.*

The paper explicitly states that control points determine the overall shape and position of the smooth curve. `ba5be526-46d7-43db-9755-56877eaf09de.pdf`

---

2. Knots

Knots are NOT points in space.

They are numbers in the parameter domain.

Example knot vector:

```
U = [0,0,0,0,1,2,3,4,4,4,4]
```

These values live on the parameter axis:

```
u

0----1----2----3----4
```

The UAV never flies to these knot values.

They simply tell the spline:

> *"How much influence should each control point have in different regions?"*

The paper describes the knot vector as partitioning the parameter domain and defining how local spline segments combine. `ba5be526-46d7-43db-9755-56877eaf09de.pdf` `ba5be526-46d7-43db-9755-56877eaf09de.pdf`

---

3. Easy Analogy

Imagine drawing a road.

Control Points

Are the pegs:

```
o-----o------o------o
```

that define the road's general shape.

Knots

Are instructions telling you:

```
Use peg 1 a lot here
Use peg 2 a little here
Switch influence at this location
```

So:

```
Control Points = WHERE the curve goes

Knots = HOW the curve transitions
```

---

4. What happens if you change only control points?

Keep knots fixed.

```
Knots:
[0,0,0,0,1,2,3,4,4,4,4]
```

Move a control point upward.

Result:

```
Curve shape changes
```

but the flexibility distribution remains the same.

---

5. What happens if you change only knots?

Keep control points fixed.

Example:

Uniform knots

```
[0,1,2,3,4,5,6]
```

Equal spacing.

Influence changes smoothly.

Dense knots near a turn

```
[0,1,2,2.1,2.2,2.3,5,6]
```

Now the spline gets more flexibility around that region.

The curve can better follow sharp maneuvers.

This is exactly what the paper exploits using a Genetic Algorithm. Instead of manually choosing knots, it optimizes them automatically. `ba5be526-46d7-43db-9755-56877eaf09de.pdf`

---

6. Why does the paper optimize knots?

Suppose the UAV path looks like:

```
----------------
               /
              /
             /
```

There is a sharp turn.

If knots are uniformly distributed:

```
0 1 2 3 4 5 6 7
```

the spline may smooth the corner too much.

But if more knots are placed near the turn:

```
0 1 2 3 3.1 3.2 3.3 4 5 6
```

the spline gains extra flexibility there.

Result:

```
Better tracking
Lower error
```

without adding complexity everywhere.

---

7. In the Paper

The optimization variables are:

Control Points

$$D = \{d_0, d_1, \ldots, d_m\}$$

These define the curve's shape. `ba5be526-46d7-43db-9755-56877eaf09de.pdf`

Knot Vector

$$U = \{u_0, u_1, \ldots, u_{m+k+1}\}$$

These define where spline segments begin/end and how much local flexibility exists. `ba5be526-46d7-43db-9755-56877eaf09de.pdf`

The paper:

1. Uses a GA to optimize **knots**
2. Uses local refinement to optimize **control points**

So both are adjusted, but for different reasons. `ba5be526-46d7-43db-9755-56877eaf09de.pdf`

Interview Answer

If someone asks:

"What is the difference between control points and knots in a B-Spline?"

You can answer:

> *Control points are geometric points that determine the shape and location of the spline curve, while knots are parameter values that determine how the basis functions are distributed and how strongly different control points influence different regions of the curve. Moving a control point changes the curve's shape, whereas changing knots changes the curve's flexibility and local behavior without moving the control points themselves.*

A useful next step is understanding **how knot vectors generate B-Spline basis functions using the Cox-de Boor recursion formula**, because that is where the role of knots becomes mathematically clear.

8/15/2026, 5:17:08 PM

0602545d-34e1-4c8c-ade0-54...

now similarly tell me about this paper

8/15/2026, 5:28:50 PM

Thinking

- **Accessed uploaded files and read the second PDF's opening pages**

✓ Worked for 4s

This paper is actually one of the best introductory papers for understanding **why drones use quaternions instead of Euler angles**.

The title is:

"Full Quaternion Based Attitude Control for a Quadrotor" by Emil Fresk and George Nikolakopoulos. `0602545d-34e1-4c8c-ade0-54f3b41e553b.pdf`

---

1. What problem is the paper solving?

A quadrotor has two major control problems:

Position Control

Move drone from A → B

Attitude Control

Control:

- Roll (φ)
- Pitch (θ)
- Yaw (ψ)

The paper focuses ONLY on attitude control. `0602545d-34e1-4c8c-ade0-54f3b41e553b.pdf`

Imagine a drone:

```
      ^
      |
      |
      o
```

If wind pushes it:

```
    /
   /
  o
```

The controller must rotate the drone back.

The question is:

> *How should we mathematically represent orientation?*

Most controllers use Euler angles.

This paper says:

> *Use quaternions directly.*

---

2. Why Euler Angles Are Bad

Euler angles are:

```
Roll  (φ)
Pitch (θ)
Yaw   (ψ)
```

Example:

```
Roll  = 30°
Pitch = 20°
Yaw   = 90°
```

Easy for humans.

But they have a major problem:

Gimbal Lock

When pitch approaches ±90°

two rotation axes align.

You lose a degree of freedom.

The paper explains this as a geometrical singularity where two rotational axes lock together. `0602545d-34e1-4c8c-ade0-54f3b41e553b.pdf`

Imagine:

```
Yaw
 ^
 |
 |
 +----> Roll
```

At certain orientations:

```
Yaw == Roll
```

Now the drone cannot distinguish some rotations.

Control becomes unstable.

---

3. Alternative Representations

Paper discusses:

Option 1

Euler Angles

Problems:

- singularities
- gimbal lock

Option 2

Direction Cosine Matrix (DCM)

Advantages:

- no gimbal lock

Problems:

- 9 states
- expensive computations

Option 3

Quaternion

Advantages:

- no singularities
- only 4 states
- computationally efficient

This is why the authors choose quaternions. `0602545d-34e1-4c8c-ade0-54f3b41e553b.pdf`

---

4. What is a Quaternion?

A quaternion is:

$$q = q_0 + q_1 i + q_2 j + q_3 k$$

or

$$q = [q_0, q_1, q_2, q_3]^T$$

`0602545d-34e1-4c8c-ade0-54f3b41e553b.pdf`

Think of it as:

```
Scalar Part
    +
Vector Part
```

```
q = [q0 q1 q2 q3]
```

where:

```
q0 -> scalar

q1
q2 -> vector
q3
```

---

5. Why Quaternions?

A rotation can be represented as:

$$q = \cos(\alpha/2) + u\sin(\alpha/2)$$

where:

- α = rotation angle
- u = rotation axis

`0602545d-34e1-4c8c-ade0-54f3b41e553b.pdf`

Example:

Rotate:

```
90°
```

around

```
z-axis
```

Quaternion stores this compactly.

---

6. Quaternion Multiplication

Suppose:

```
Rotation A
```

and

```
Rotation B
```

Quaternion multiplication:

$$A \otimes B$$

combines them.

The paper calls this the Kronecker product notation. `0602545d-34e1-4c8c-ade0-54f3b41e553b.pdf`

Important:

$$A \otimes B \neq B \otimes A$$

Just like real rotations. `0602545d-34e1-4c8c-ade0-54f3b41e553b.pdf`

---

7. Main Contribution of the Paper

Earlier quaternion controllers did:

```
Quaternion
     ↓
Euler Angles
     ↓
Control
```

Meaning:

They converted back to Euler angles.

The paper argues this reintroduces:

- singularities
- nonlinearities
- extra computations

`0602545d-34e1-4c8c-ade0-54f3b41e553b.pdf`

Their idea:

```
Quaternion
      ↓
Control
      ↓
Torque
```

No Euler conversion at all. `0602545d-34e1-4c8c-ade0-54f3b41e553b.pdf`

---

8. Quadrotor Model

State variables:

Orientation

Quaternion

```
q
```

Angular Velocity

```
ωx
ωy
ωz
```

The system dynamics become:

$$\dot{q} = f(q, \omega)$$

$$\dot{\omega} = g(\omega, \tau)$$

where:

```
τ = torque
```

generated by motors. `0602545d-34e1-4c8c-ade0-54f3b41e553b.pdf`

---

9. Error Calculation

Suppose:

Current orientation:

```
qm
```

Desired orientation:

```
qref
```

Error quaternion:

$$q_{err} = q_{ref} \otimes q_m^*$$

where:

```
* = conjugate
```

`0602545d-34e1-4c8c-ade0-54f3b41e553b.pdf`

This is one of the most important equations in drone attitude control.

---

10. Why This Error Works

Imagine:

Current:

```
0°
```

Desired:

```
90°
```

The error quaternion directly tells:

```
Which axis?
How much rotation?
```

without Euler angles.

---

11. Extracting Axis Error

The controller only uses:

$$[q_1, q_2, q_3]$$

the vector part.

$$Axis_{err} = [q_{err1}, q_{err2}, q_{err3}]$$

`0602545d-34e1-4c8c-ade0-54f3b41e553b.pdf`

Think:

```
q1 -> roll error
q2 -> pitch error
q3 -> yaw error
```

(not exactly Euler errors, but conceptually similar).

---

12. The Proposed Controller

This is the heart of the paper.

They propose:

P² Controller

Meaning:

```
Outer P controller
+
Inner P controller
```

The control law:

$$\tau = -P_q \begin{bmatrix} q_1 \\ q_2 \\ q_3 \end{bmatrix} - P_\omega \begin{bmatrix} \omega_x \\ \omega_y \\ \omega_z \end{bmatrix}$$

`0602545d-34e1-4c8c-ade0-54f3b41e553b.pdf`

Interpretation:

First term

Orientation correction

```
You are tilted
```

Second term

Angular velocity damping

```
You are spinning too fast
```

Together:

```
Rotate toward target
+
Slow down oscillations
```

---

13. Why Is It Called P²?

Two proportional loops:

```
Quaternion Error
      ↓
      P
      ↓
Angular Velocity Ref
      ↓
      P
      ↓
Torque
```

Hence:

```
P × P = P²
```

The block diagram in the paper shows exactly this architecture. `0602545d-34e1-4c8c-ade0-54f3b41e553b.pdf`

---

14. Why Is It Good?

Advantages:

No Gimbal Lock

Works for full 360° rotations.

No Trigonometric Functions

Less computation.

No Euler Conversion

Simpler implementation.

Low Cost

Only proportional gains.

The paper explicitly highlights derivative-free implementation and low computational cost. `0602545d-34e1-4c8c-ade0-54f3b41e553b.pdf`

---

15. Simulations

They tested three cases.

Test 1: Step Response

Command:

```
Rotate 1 radian
```

The drone quickly reached the target.

Small overshoot.

Stable behavior. `0602545d-34e1-4c8c-ade0-54f3b41e553b.pdf`

Test 2: Sinusoidal Tracking

Reference:

```
sin(t)
```

Drone continuously follows changing attitude.

Result:

Good tracking.

Small phase lag (~0.5 s). `0602545d-34e1-4c8c-ade0-54f3b41e553b.pdf`

Test 3: Full Flip

Most interesting experiment.

Command:

```
0 → 360°
```

A complete flip.

Euler-angle controllers often struggle near singularities.

Quaternion controller handled it naturally. `0602545d-34e1-4c8c-ade0-54f3b41e553b.pdf`

---

16. Biggest Insight

The entire paper can be summarized as:

Traditional approach:

```
Quaternion
    ↓
Euler
    ↓
Controller
```

Proposed approach:

```
Quaternion
    ↓
Controller
```

No intermediate representation. `0602545d-34e1-4c8c-ade0-54f3b41e553b.pdf`

---

17. Limitations

As a reviewer, I would note:

❌ Only simulation results

❌ No real flight experiments

❌ Simple P^2 controller

❌ No comparison with modern methods:

- Sliding Mode Control
- Backstepping
- MPC
- Adaptive Control
- Reinforcement Learning

❌ Gains tuned manually

Research-Level Takeaway

This paper is not mainly about inventing a powerful controller.

Its real contribution is:

> *Represent the entire quadrotor attitude dynamics and control loop directly in quaternion space, eliminating Euler-angle singularities and reducing computational complexity while maintaining stable attitude tracking.* `0602545d-34e1-4c8c-ade0-54f3b41e553b.pdf`

Since you recently asked about **gimbal lock, quaternions, Hamilton products, and the sandwich operator**, this paper is essentially the practical drone-control application of those concepts. The next concept to master is:

Quaternion Error = $q_{ref} \otimes q_{current}^{-1}$

because that single equation is the foundation of most modern drone attitude controllers.

8/15/2026, 5:28:52 PM

cc1e367f-634a-472e-8b27-657...

what my faculty was to combine both the papers, like do path things with paper 1 and the control things with paper 2.

And my teammate have done something by combing both papers, is it correct, also what things need to do, like where to simulate , how , which tools to use, what features can i add to existing things or is there anything to change , upgrade, my teammate done something , do verify everything, and givme me future works

8/15/2026, 5:36:19 PM

Thinking

- **Evaluating the repository**
  I'm checking the documented implementation against the cited papers and current simulator support. Key concerns include quaternion conventions, incomplete path-following dynamics, frame singularities, and whether the claimed planning method matches the source research.
- **Evaluated the repository**
- **Searching the web**
  mathworks.com docs.px4.io
- **Searched 2 websites**

✓ Worked for 10s

Yes - **the basic idea your faculty gave is correct**, and your teammate has made a meaningful first integration. But there is an important distinction:

> ***They have successfully connected "path geometry → desired attitude → attitude control," but they have not yet built a complete 3D trajectory-***

> ***tracking UAV system.***

That distinction is the biggest thing I would fix before you present this as a research project.

I reviewed your teammate's technical document against both papers, and I also checked current UAV-simulation/control tooling and recent B-Spline UAV work. The assessment below separates **what is correct, what is questionable, and what I would upgrade.**

---

1. First: What your faculty probably intended

The two papers solve two different layers of the UAV problem.

Paper 1

Where should the UAV fly?

It produces a smooth 3D path:

$$\mathbf{p}_d(u) = C(u)$$

using:

```
A*/other planner
      ↓
waypoints
      ↓
B-Spline
      ↓
smooth path
```

The actual Paper 1 contribution is adaptive knot selection + curvature-aware control-point refinement. `ba5be526-46d7-43db-9755-56877eaf09de.pdf`

Paper 2

How should the UAV orient itself while flying?

It controls:

$$q, \omega$$

using quaternion attitude error and the nonlinear P^2 controller:

$$\tau = -P_q q_{err,v} - P_\omega \omega$$

`0602545d-34e1-4c8c-ade0-54f3b41e553b.pdf`

So conceptually:

```
            PLANNING
               │
               ▼
        Smooth 3D B-Spline
               │
               ▼
      Desired trajectory
               │
               ▼
        Desired orientation
               │
               ▼
            CONTROL
               │
               ▼
      Quaternion P² controller
               │
               ▼
            Torque
               │
               ▼
        Quadrotor dynamics
```

That is a sensible research direction.

---

2. What your teammate has already done

According to the documentation, your teammate implemented:

```
3D Environment
      ↓
3D A*
      ↓
Waypoint simplification
      ↓
```

```
Cubic B-Spline
      ↓
GA knot optimization
      ↓
Curvature refinement
      ↓
Spline tangent
      ↓
Desired quaternion
      ↓
Quaternion P² controller
      ↓
Rigid-body rotational dynamics
      ↓
RK4
```

The documentation explicitly describes this architecture. `cc1e367f-634a-472e-8b27-657b323499fd.md`

This is **not nonsense or a superficial combination**. There is a genuine logical connection between the papers.

---

3. What is correct in their implementation?

A lot of it is good.

A. Paper 1's three-stage structure

They implemented:

Stage 1

Chord-length parameterization + control-point initialization.

Stage 2

GA-based knot optimization.

Stage 3

Local curvature refinement.

That matches the basic three-phase structure described in Paper 1. `cc1e367f-634a-472e-8b27-657b323499fd.md`

Good.

B. They correctly identified the role of knots

Their GA modifies interior knots rather than treating knots as geometric positions.

That is conceptually correct.

The documentation explicitly says the GA optimizes the interior knot vector. `cc1e367f-634a-472e-8b27-657b323499fd.md`

C. They correctly use curvature

They use:

$$\kappa(u) = \frac{\|C'(u) \times C''(u)\|}{\|C'(u)\|^3}$$

which is the correct parametric curvature expression used by Paper 1. `cc1e367f-634a-472e-8b27-657b323499fd.md`

D. The quaternion error calculation is conceptually correct

They use:

$$q_{err} = q_{ref} \otimes q_{meas}^{-1}$$

This agrees with Paper 2. `0602545d-34e1-4c8c-ade0-54f3b41e553b.pdf`

E. The P² controller is correctly represented

They use:

$$\tau = -P_q q_{err,v} - P_\omega \omega$$

which corresponds to Paper 2's control law. `0602545d-34e1-4c8c-ade0-54f3b41e553b.pdf`

F. RK4 is a sensible improvement

The paper presents continuous dynamics; using RK4 for numerical integration is perfectly reasonable.

Your teammate also renormalizes the quaternion, which is important because numerical integration can cause drift from:

$$\|q\| = 1$$

The documentation confirms this. `cc1e367f-634a-472e-8b27-657b323499fd.md`

---

4. But there is a BIG problem

This is the most important point.

Your teammate currently has:

$$\boxed{\text{Path tangent} \rightarrow q_{ref} \rightarrow \text{attitude control}}$$

but **not**

$$\boxed{\text{Path position} \rightarrow \text{position control} \rightarrow \text{thrust} + \text{attitude}}$$

These are not the same thing.

---

5. Why tangent → quaternion is insufficient

Suppose your B-Spline is:

```
            *
         *
      *
   *
*
```

The tangent tells you:

```
Which direction should the drone face?
```

But it does NOT tell the drone:

```
Where should the drone be?
How fast should it move?
How much acceleration is required?
How much thrust is required?
```

Your teammate's controller only controls:

$$q, \omega, \tau$$

It doesn't control:

$$x, y, z$$

or:

$$v_x, v_y, v_z$$

or total thrust.

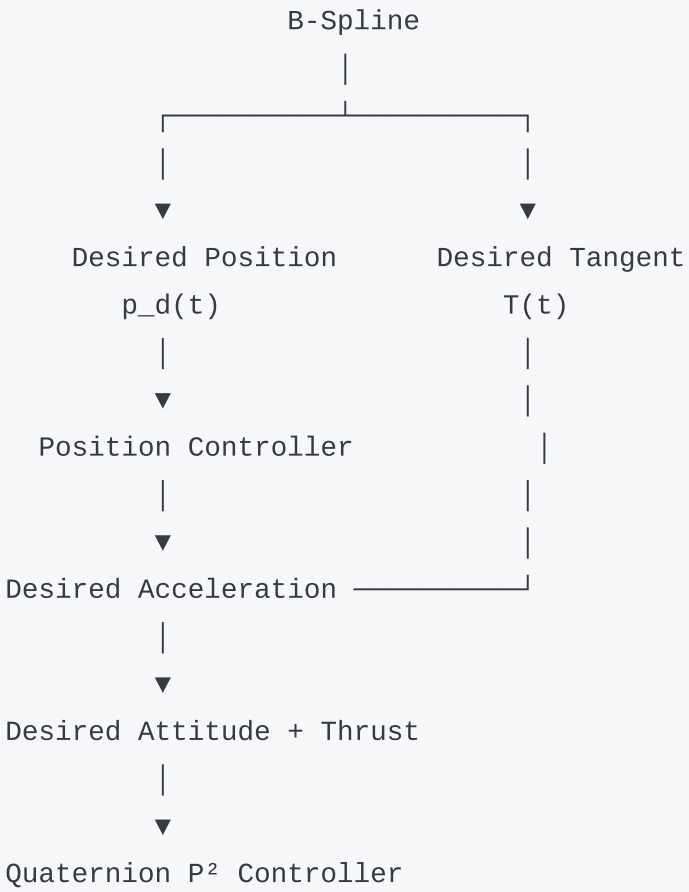
The current architecture therefore represents an **attitude-following layer**, not a full trajectory-tracking controller.

This is consistent with Paper 2 itself: it explicitly addresses the attitude problem rather than the full translational navigation problem. `0602545d-34e1-4c8c-ade0-54f3b41e553b.pdf`

---

6. This is the change I would make

Your system should become:



```
          |
          ▼
Motor Torques / Forces
          |
          ▼
 6-DOF Quadrotor Dynamics
          |
          ▼
Actual Position + Attitude
          |
          └──────────────► feedback
```

That would be a much stronger combination of the two papers.

---

7. In other words, add a position controller

For example:

$$e_p = p_d - p$$

$$e_v = v_d - v$$

Then:

$$a_d = K_p e_p + K_v e_v + a_{ff}$$

where:

- p_d = B-Spline desired position
- p = current position
- v_d = desired velocity
- v = current velocity
- a_{ff} = feed-forward acceleration from the spline

This is where the project becomes much more interesting.

---

8. The B-Spline gives you much more than position

This is a major opportunity for your project.

You already have:

$$C(u)$$

Differentiate it:

$$C'(u)$$

$$C''(u)$$

$$C'''(u)$$

You can obtain:

Position

$$p_d = C(u)$$

Velocity

$$v_d = C'(u)\dot{u}$$

Acceleration

$$a_d = C''(u)\dot{u}^2 + C'(u)\ddot{u}$$

Jerk

$$j_d = C'''(u)\dot{u}^3 + 3C''(u)\dot{u}\ddot{u} + C'(u)\dddot{u}$$

Now the path planner and controller are **actually coupled mathematically**.

That's much stronger than simply taking the tangent.

---

9. Another issue: the teammate's "gated path progress"

They use:

$$u_{k+1} = \begin{cases} u_k + \Delta u & \|q_{err}\| < \epsilon \\ u_k & otherwise \end{cases}$$

The documentation calls this a gated guidance loop. `cc1e367f-634a-472e-8b27-657b323499fd.md`

This is an interesting engineering idea, but:

It is NOT from either paper.

It is their integration design.

That's okay.

But don't write:

> *"The papers propose gated path progression."*

They don't.

Present it as:

> *"We introduce an attitude-convergence-based path-progress mechanism."*

That can potentially become one of your contributions.

However, I wouldn't use attitude error alone as the progress condition.

Better:

$$\|p - p_d\| < \epsilon_p$$

AND

$$\|v - v_d\| < \epsilon_v$$

AND

$$\|q_{err}\| < \epsilon_q$$

Then advance the path.

That prevents the UAV from moving forward along the reference path simply because its orientation is correct while its actual position is still far away.

---

10. BIGGER issue: the "tangent → quaternion" formulation

Your teammate constructs:

$$X_b = T$$

and uses a reference up vector to construct the other axes. `cc1e367f-634a-472e-8b27-657b323499fd.md`

This can work for a heading-oriented representation, but there is an important singular case.

They use:

$$Z_{ref} = [0, 0, 1]^T$$

and:

$$Y_b = \frac{Z_{ref} \times X_b}{\|Z_{ref} \times X_b\|}$$

What happens when:

$$X_b \approx Z_{ref}$$

?

The cross product approaches zero.

You have another degeneracy.

So I would modify the frame construction.

Use an adaptive reference vector:

```
if |T · Zref| < threshold:
       use Zref
else:
       use Xref or another orthogonal vector
```

or construct the attitude from the desired acceleration/thrust direction instead.

That is a much better quadrotor formulation.

---

11. More important: body X or body Z?

This needs to be explicitly defined.

The teammate is effectively saying:

```
Body X axis = path tangent
```

That's a valid orientation convention **if your intended heading is along the path**.

But a quadrotor generates its primary thrust along its body axis, depending on the chosen frame/sign convention.

Therefore, for a real quadrotor trajectory controller, you normally want the desired acceleration/thrust vector to determine the appropriate thrust axis, while yaw can be selected separately.

I would therefore define:

$$b_{3,d} = -\frac{a_d - g}{\|a_d - g\|}$$

or the equivalent sign depending on your coordinate/thrust convention.

Then choose desired yaw:

$$\psi_d = \operatorname{atan2}(T_y, T_x)$$

and construct R_d.

That gives you a physically meaningful desired attitude.

---

12. Very important: check quaternion convention

There is something in the teammate's documentation that **must be checked in the actual code**.

They document:

$$\dot{q} = \frac{1}{2} q \otimes \begin{bmatrix} 0 \\ \omega \end{bmatrix}$$

`cc1e367f-634a-472e-8b27-657b323499fd.md`

But Paper 2's final quadrotor rotational model is written as:

$$\dot{q} = -\frac{1}{2} \begin{bmatrix} 0 \\ \omega \end{bmatrix} \otimes q$$

`0602545d-34e1-4c8c-ade0-54f3b41e553b.pdf`

These can be related through convention choices, but **you cannot casually mix left/right quaternion multiplication, sign convention, body-vs-world angular velocity, and frame direction.**

So this has to be checked in the actual `quaternion.py` and `quadrotor_model.py`.

The markdown alone is not enough to certify that implementation.

My verdict:

Do not change it blindly.

First establish:

```
Quaternion convention:
Hamilton?
scalar-first?

Angular velocity:
body frame or inertial frame?

q maps:
body → world or world → body?

Kinematic equation:
left or right multiplication?

Rotation matrix:
active or passive?
```

Then use exactly one convention throughout the project.

This is one of the first things I would test.

---

13. Another issue: their Paper 1 objective is incomplete

This is actually documented very clearly.

Paper 1's objective includes:

$$J = w_1 F_{data} + w_2 F_{curv} + w_3 F_{dyn} + w_4 F_{len} + w_5 F_{knot}$$

`ba5be526-46d7-43db-9755-56877eaf09de.pdf`

But the teammate explicitly says:

> F_{dyn} *is omitted.*

Their implementation uses a 4-term GA cost. `cc1e367f-634a-472e-8b27-657b323499fd.md`

This is a weakness.

I would fix this.

Add:

$$F_{dyn} = \lambda_v \int \max(0, \|v\| - v_{max})^2 du + \lambda_a \int \max(0, \|a\| - a_{max})^2 du$$

Potentially also:

$$+\lambda_j \int \|j\|^2 du$$

Now the path itself becomes dynamically meaningful.

---

14. Do NOT claim "Paper 1 path planning" too strongly

This is an important terminology issue.

Paper 1 is primarily:

> ***adaptive B-Spline smoothing***

It assumes/generated raw track points and then smooths them. It discusses the usual global-planner + local-smoothing architecture. `ba5be526-46d7-43db-9755-56877eaf09de.pdf`

Your teammate added:

```
3D A*
```

That's reasonable, but **A*** wasn't the contribution of Paper 1.

So your thesis/project should say something like:

> *"A 3D A* global planner is used to generate collision-free waypoints, which are subsequently smoothed using the adaptive GA-optimized B-Spline framework."*

That is scientifically accurate.

---

15. Their numerical result is also suspicious

Their recreated numbers are:

Method	Length	Max curvature
Baseline Polyline	2984.6	0.650
Uniform B-Spline	3065.4	0.007
Classical B-Spline	2803.2	0.049
Adaptive	2897.2	0.0297

`cc1e367f-634a-472e-8b27-657b323499fd.md`

But the original Paper 1 reports:

Method	Path Length	Max Curvature
Baseline	2984.6	0.65
Uniform	2840.8	0.56
Classical	2803.2	0.49
Proposed	2731.4	0.38

`ba5be526-46d7-43db-9755-56877eaf09de.pdf`

So your teammate's recreated results **do not reproduce Paper 1's Table 2**.

That's not automatically bad.

But they absolutely should not say:

> *"We reproduced Paper 1's results."*

Instead:

> *"Our independent implementation produced different numerical results under our implementation/setup."*

Then investigate why.

Possible causes include:

- different waypoints

- different parameterization
- different knot count
- different curvature sampling
- different scaling
- different GA parameters
- different control-point fitting
- different obstacle configuration
- different path-generation procedure

This is actually a useful reproducibility experiment.

---

16. Another issue: their curvature values changed in an unintuitive direction

Their baseline:

$$0.007$$

GA:

$$0.0297$$

So the GA apparently **increased maximum curvature** while reducing path length.

That could be completely legitimate because their objective is multi-objective.

But then the terminology needs to be precise.

The algorithm isn't simply:

> *"minimizing curvature."*

It is trading:

```
path fidelity
+
curvature
+
length
```

```
+
knot regularity
```

Your plots should therefore show **Pareto/weighted trade-offs**, not only "lower is better."

---

17. I would change the project architecture

Here's the version I recommend.

Layer 1 - Environment

```
3D map
│
├── static obstacles
├── dynamic obstacles
└── wind/disturbance model
```

Layer 2 - Global planner

```
A*
RRT*
or kinodynamic search
```

Output:

$$P_0, P_1, \ldots, P_n$$

Layer 3 - Adaptive B-Spline

```
waypoints
   ↓
chord parameterization
   ↓
control points
   ↓
GA knot optimization
   ↓
curvature refinement
```

```
    ↓
C(u)
```

Layer 4 - Time parameterization

This is currently missing.

You need:

$$u(t)$$

rather than just:

$$u$$

Then obtain:

$$p_d(t), v_d(t), a_d(t), j_d(t)$$

This is critical.

Layer 5 - Position controller

For example:

$$e_p = p_d - p$$

$$e_v = v_d - v$$

$$a_c = a_d + K_p e_p + K_v e_v$$

Layer 6 - Desired attitude generator

From desired acceleration:

$$a_c$$

construct desired body orientation:

$$R_d$$

then:

$$R_d \to q_d$$

Layer 7 - Paper 2 quaternion controller

$$q_e = q_d \otimes q^{-1}$$

$$\tau = -K_q q_{e,v} - K_\omega \omega$$

Layer 8 - Full 6-DOF dynamics

Not just:

$$\dot{q}, \dot{\omega}$$

but:

$$\dot{p} = v$$

$$m\dot{v} = F + mg$$

$$\dot{q} = f(q, \omega)$$

$$I\dot{\omega} = \tau - \omega \times (I\omega)$$

Now you're simulating an actual moving quadrotor.

---

18. What simulator should you use?

You have three good choices.

Option A - MATLAB/Simulink

Best for your academic project

Especially because your faculty is combining trajectory planning and control.

MATLAB/Simulink's current UAV tooling supports:

- UAV modeling
- PX4
- SITL
- HITL
- controller design
- quadrotor plant simulation

MathWorks currently provides examples specifically for **quadrotor position control and PX4 SITL**. (MathWorks)

This would be my **first choice** for the research paper.

---

19. Option B - PX4 + Gazebo

This is better if you want actual robotics credibility.

Architecture:

```
Your planner
      ↓
ROS 2 / MAVLink
      ↓
PX4
      ↓
Gazebo
      ↓
Quadrotor
```

Current PX4 documentation supports Gazebo Harmonic SITL, including the X500 quadrotor. (PX4 Docs)

This gives you a route toward:

```
simulation
   ↓
SITL
   ↓
HITL
   ↓
real Pixhawk
```

---

20. Option C - Python

Python is excellent for:

```
B-Spline
GA
path optimization
research experiments
plotting
```

Your teammate's Python implementation is therefore useful.

But I wouldn't make Python alone your final validation platform.

Use Python for:

> *algorithm development + ablation studies*

and MATLAB/PX4/Gazebo for:

> *realistic UAV validation.*

---

21. My recommended toolchain for you

I'd use:

```
Python
│
├── NumPy
├── SciPy
├── PyGAD
├── Matplotlib
└── optimization/analysis
          │
          ▼
MATLAB / Simulink
│
├── 6-DOF quadrotor model
├── position controller
├── quaternion controller
└── validation
          │
          ▼
PX4 SITL + Gazebo
│
└── realistic UAV simulation
```

Later:

```
PX4 SITL
   ↓
```

```
HITL
   ↓
Pixhawk
```

MathWorks currently documents both SITL and HITL workflows, including PX4 Host Target and Pixhawk hardware. (MathWorks)

---

22. What experiments should you perform?

This is where you can turn this from a student implementation into a proper research project.

Experiment 1 - Path quality

Compare:

```
A* raw
Uniform B-Spline
Chord-length B-Spline
GA adaptive B-Spline
```

Measure:

$$L$$

$$\kappa_{max}$$

$$SR$$

Experiment 2 - Dynamic feasibility

Measure:

$$v_{max}$$

$$a_{max}$$

$$j_{max}$$

This is currently missing.

Experiment 3 - Tracking performance

Compare:

```
No attitude controller
PID attitude
Paper 2 P²
P² + your improvements
```

Measure:

$$RMSE_{position}$$

$$RMSE_{attitude}$$

$$RMSE_{velocity}$$

---

23. Experiment 4 - Disturbance robustness

Apply:

Wind

```
0 m/s
2 m/s
5 m/s
8 m/s
```

Sensor noise

GPS/IMU noise.

Model uncertainty

Change:

$$I_{xx}, I_{yy}, I_{zz}$$

by ±10-20%.

Then measure whether the drone remains stable.

This is directly relevant because Paper 2 motivates robustness to external disturbances, while its actual evaluation remains simulation based. `0602545d-34e1-4c8c-ade0-54f3b41e553b.pdf`

24. Experiment 5 - Dynamic obstacles

The original Paper 1 explicitly identifies dynamic obstacles as a limitation/future direction. `ba5be526-46d7-43db-9755-56877eaf09de.pdf`

So introduce:

```
Static building
       +
Moving drone
       +
Moving vehicle
```

Then trigger local replanning.

This would make your project substantially stronger.

---

25. Experiment 6 - GA computation time

Measure:

$$T_{GA}$$

against:

```
population size
generation count
number of knots
number of waypoints
```

You want to answer:

> *Can this run fast enough for online replanning?*

Paper 1 itself acknowledges the computational cost of GA. `ba5be526-46d7-43db-9755-56877eaf09de.pdf`

---

26. Experiment 7 - Ablation study

This is **very important for your research paper**.

Run:

A

B-Spline only

B

B-Spline + GA

C

B-Spline + GA + curvature refinement

D

Full system

E

Full system + position controller

F

Full system + disturbance handling

Then you can prove where every contribution helps.

---

27. What could be your research contribution?

Simply saying:

> *"We combined Paper 1 and Paper 2"*

is probably not enough for a strong research contribution.

The combination itself is the starting point.

A stronger contribution could be:

Contribution 1

Dynamics-aware adaptive B-Spline

Add velocity, acceleration and jerk into GA optimization.

Contribution 2

Trajectory-to-attitude transformation

Instead of using only tangent:

$$C'(u)$$

use:

$$C'(u), C''(u), C'''(u)$$

to generate physically meaningful trajectory references.

Contribution 3

Position + quaternion attitude cascade

Add:

```
position controller
        ↓
desired acceleration
        ↓
desired attitude
        ↓
quaternion P²
```

This is probably the biggest improvement I'd make.

Contribution 4

Adaptive path progression

Instead of fixed:

$$\Delta u$$

make it dependent on tracking quality.

For example:

$$\dot{u} = f(e_p, e_v, e_q)$$

The UAV slows down in difficult turns and speeds up in easy segments.

That is a very natural connection between planning and control.

---

28. This one could be particularly interesting

Imagine the UAV reaches a sharp turn.

Your B-Spline says:

```
high curvature
```

Your controller knows:

```
high attitude demand
```

Instead of flying at the same speed, you reduce velocity:

$$v_d = f(\kappa)$$

For example conceptually:

$$v_d = \frac{v_{max}}{1 + \lambda \kappa}$$

So:

```
Straight path
κ low
   ↓
fast

Sharp turn
κ high
   ↓
slow
```

This creates a direct relationship:

$$\boxed{\text{B-Spline curvature} \rightarrow \text{speed planning} \rightarrow \text{attitude control}}$$

That is much more interesting than simply connecting two existing papers.

---

29. Another strong extension: dynamic knot placement + time allocation

Current Paper 1 optimizes:

```
where knots are placed
```

But the UAV also needs:

```
how quickly to traverse each segment
```

So optimize both:

$$U$$

and

$$T$$

where T is segment duration.

Then your optimization becomes:

$$J(U,T) = w_1 F_{tracking} + w_2 F_{curvature} + w_3 F_{velocity} + w_4 F_{acceleration} + w_5 F_{jerk} + w_6 F_{time}$$

This would connect planning directly to control.

---

30. Recent research tells you this is the direction to pursue

Current UAV research is already moving beyond simply smoothing a path.

For example, recent work combines B-Splines with dynamic environments, safety corridors and dynamic feasibility rather than treating smoothing as an isolated post-processing step. (MDPI)

Other current work combines B-Spline trajectory generation with search, optimization, dynamic constraints and even real flight experiments. (MDPI)

There is also recent work coupling B-Spline planning with online/local control and disturbance handling. (HKUST Research Portal)

So I would **not** position your novelty as simply "B-Spline + quaternion controller." That combination alone is unlikely to be sufficiently novel.

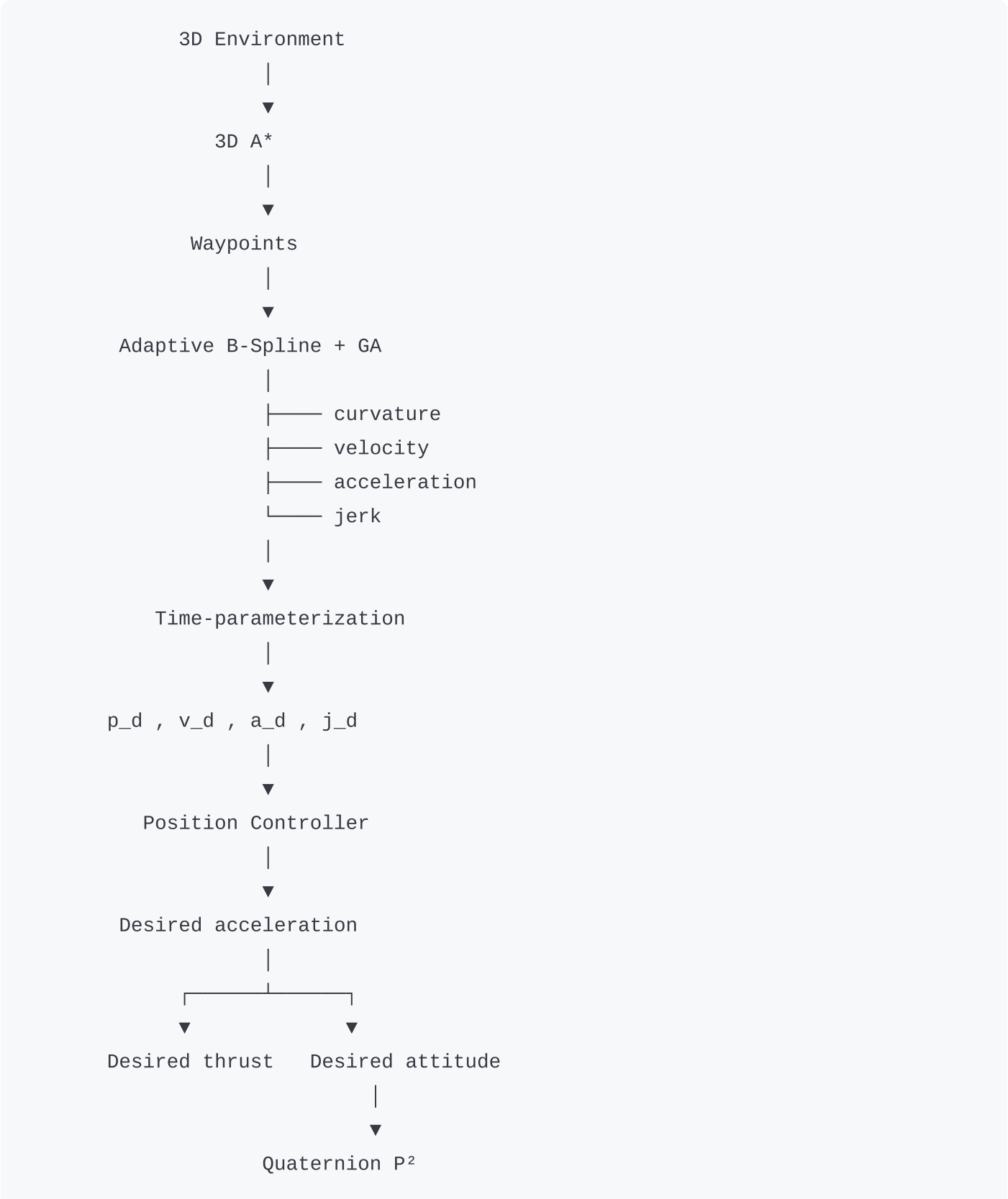
---

31. My recommended final project

I would make the project:

Adaptive Dynamics-Aware B-Spline Trajectory Planning and Quaternion-Based 6-DOF Control for Robust Quadrotor Navigation

Architecture:



```
                    │
                    ▼
              6-DOF dynamics
                    │
                    ▼
               Actual UAV
                    │
                    └──── feedback
```

That is the system I would build.

---

32. What I would keep from your teammate

Keep:

✅ 3D A*

✅ cubic B-Spline

✅ chord-length parameterization

✅ GA knot optimization

✅ curvature refinement

✅ quaternion mathematics

✅ quaternion error

✅ P^2 controller

✅ RK4

✅ obstacle environment

✅ Python implementation

✅ MATLAB implementation

These are good foundations. `cc1e367f-634a-472e-8b27-657b323499fd.md`

---

33. What I would change

Change 1 - Add translational dynamics

Currently missing.

Change 2 - Add position controller

Essential.

Change 3 - Add thrust computation

Essential for a real quadrotor.

Change 4 - Add time parameterization

Very important.

Change 5 - Add velocity/acceleration/jerk to GA objective

Directly addresses the missing F_{dyn}. `cc1e367f-634a-472e-8b27-657b323499fd.md`

Change 6 - Replace attitude-only gating with position + velocity + attitude convergence

Much more physically meaningful.

Change 7 - Fix/verify quaternion conventions

Critical.

Change 8 - Handle tangent/up-vector degeneracy

Critical.

Change 9 - Validate against original papers separately

Do not immediately run the combined system.

Change 10 - Add serious baselines

Especially MPC or another trajectory-tracking controller.

---

34. Future work you can put in the paper

I would structure your future work around these:

1. Dynamic obstacle avoidance

Integrate real-time obstacle detection and incremental B-Spline replanning.

Paper 1 itself identifies dynamic obstacles as an unresolved area. `ba5be526-46d7-43db-9755-56877eaf09de.pdf`

2. Wind-aware trajectory optimization

Add wind estimates into:

$$F_{dyn}$$

and obstacle safety margins.

3. Adaptive speed planning

Make:

$$v_d = f(\kappa)$$

so UAV slows automatically in high-curvature regions.

4. Full 6-DOF dynamics

Move from attitude-only simulation to:

$$x, y, z, v_x, v_y, v_z, q, \omega$$

5. Model Predictive Control

Compare P² against MPC for tracking and disturbance rejection.

6. Learning-assisted GA

Use ML/RL to initialize or tune GA parameters instead of starting from scratch.

7. Real-time optimization

GPU/parallel GA or reduced-order optimization to reduce replanning latency.

8. PX4 SITL/HITL

Deploy the controller into PX4 SITL and eventually HITL. Current MATLAB/PX4 tooling supports both SITL and HITL workflows. (MathWorks)

9. Real UAV experiments

Ultimately test on a Pixhawk-class flight controller with a real quadrotor.

10. Uncertainty-aware planning

Account for:

- GPS error
- IMU bias
- localization uncertainty
- wind
- actuator saturation
- model mismatch

---

35. My final verdict on your teammate's work

Component	Verdict
Paper 1 implementation	🟢 Good foundation
Paper 2 implementation	🟢 Good foundation
GA knot optimization	🟢 Correct concept
Curvature refinement	🟢 Good
Quaternion error	🟢 Correct concept
P^2 controller	🟢 Correct concept
RK4	🟢 Good
Paper integration	🟡 Reasonable first step
Tangent → quaternion	🟡 Needs stronger formulation
Gated u progression	🟡 Interesting but custom

Component	Verdict
Full trajectory tracking	🔴 Missing
Position control	🔴 Missing
Thrust control	🔴 Missing
6-DOF dynamics	🔴 Missing
Dynamic feasibility cost	🔴 Missing
Exact reproduction of Paper 1 numbers	🔴 Not achieved
Research novelty	🟡 Not enough yet
Potential after upgrades	🟢🟢 Very good

So **don't throw away your teammate's work**. It is actually a useful starting point.

But I would **not submit the current version as the final combined research system**. The biggest upgrade is to move from:

> ***"smooth path → desired orientation → attitude controller"***

to:

> ***"smooth, time-parameterized, dynamics-aware trajectory → position/velocity controller → desired thrust + quaternion attitude → full 6-DOF quadrotor dynamics."***

That is the direction I would take.

One important limitation of my verification: I verified the teammate's architecture and reported results against the two papers using the documentation you uploaded, but I **did not have the actual Python/MATLAB source files in this conversation**, so I cannot honestly certify the implementation-level details such as the exact quaternion multiplication convention, frame convention, RK4 equations, or A* collision logic. The documentation itself identifies those areas clearly enough to know what should be audited. `cc1e367f-634a-472e-8b27-657b323499fd.md`

Sources

UAV Toolbox Support Package for PX4 Autopilots - MATLAB & Simulink mathworks.com

Gazebo Simulation | PX4 Guide (v1.17) docs.px4.io

Quadrotor Trajectory Planning with Tetrahedron Partitions and B-Splines in Unknown and Dynamic Environments mdpi.com

Robust and Efficient Quadrotor Trajectory Generation for Fast Autonomous Flight - The Hong Kong University of Science and Technology Research Portal researchportal.hkust.edu.hk

8/15/2026, 5:36:22 PM

Pasted markdown(20260815-1...

ok, these are very great but first what i will do is, i will combine both and implement what is done already in the paper. then i will do extended work. so give me a plan. As this was done partially , tell me was this correct and what i need to do next

8/15/2026, 8:08:06 PM

Yes. **That is the right strategy.** Don't jump into extensions yet.

First make a clean, defensible **baseline that faithfully implements both papers and connects them**. Once that works, freeze it. Only then start your extended research.

I reviewed the latest project documentation you uploaded. At the architecture level, your teammate has already done a substantial part of the work, but there are a few things that should be verified/corrected before you call it a faithful implementation.
`Pasted markdown.md`

1. Where your project is right now

Your teammate has essentially completed:

```
3D Environment
      ↓
3D A*
      ↓
Waypoints
      ↓
Cubic B-Spline
      ↓
GA Knot Optimization
      ↓
Curvature Refinement
      ↓
Spline Tangent
```

```
      ↓
Desired Quaternion
      ↓
Quaternion P² Controller
      ↓
Rotational Dynamics
      ↓
RK4
```

That is a **reasonable first integration of the two papers**. The documentation confirms all of these modules exist in the planned implementation. `Pasted markdown.md`

So **do not rebuild everything**.

Your immediate goal should be:

> ***Validate every block independently, then validate the connection between blocks.***

---

2. Very important: don't call the current system "finished"

There are three different levels here:

Level 1 - Paper 1 reproduction

Can we reproduce adaptive B-Spline smoothing?

Level 2 - Paper 2 reproduction

Can we reproduce quaternion attitude control?

Level 3 - Combined system

Can Paper 1's output meaningfully drive Paper 2?

Your teammate has partially reached Level 3, but Levels 1 and 2 need to be verified independently first.

This is the correct research workflow.

---

3. Your plan from now

I would divide your work into **6 phases**.

PHASE 1 - Freeze the current implementation

Do not add any new feature yet.

Create a branch/version:

```
baseline-paper-combination
```

Keep the current implementation untouched.

The repository already has separate modules for:

- A*
- B-Spline
- GA
- curvature refinement
- quaternion
- quadrotor rotational model
- P² controller
- integration loop

according to the documentation. `Pasted markdown.md`

This is good because you can test each part independently.

PHASE 2 - Reproduce Paper 1 first

Before involving the quaternion controller, run:

```
Waypoints
    ↓
Uniform B-Spline
    ↓
Classical B-Spline
    ↓
Adaptive B-Spline
```

and calculate exactly:

$$L$$

$$\kappa_{\max}$$

$$SR$$

The original Paper 1 compares four approaches and reports these metrics. `ba5be526-46d7-43db-9755-56877eaf09de.pdf`

Your teammate already has a `paper1_recreation.py`, which is exactly the right idea. `Pasted markdown.md`

What you should do next

Run each method on the **same waypoint set**.

Generate:

```
raw waypoints
uniform spline
chord-length spline
adaptive GA spline
```

on one 3D plot.

Then produce a table:

Method	Length	Max Curvature	Straightness
Polyline			
Uniform B-Spline			
Classical B-Spline			
Adaptive B-Spline			

4. Don't worry yet that your numbers differ from the paper

Your teammate currently gets:

- Uniform: 3065.4 m
- Classical: 2803.2 m
- Adaptive: 2897.2 m

and the documentation labels these as recreated Paper 1 metrics. `Pasted markdown.md`

But Paper 1 reports different values for several methods, including:

- Uniform: 2840.8 m
- Classical: 2803.2 m
- Proposed: 2731.4 m. `ba5be526-46d7-43db-9755-56877eaf09de.pdf`

So **do not alter the code just to make the numbers match**.

Instead, first investigate:

```
Are the same waypoints used?
Same knot count?
Same parameterization?
Same sampling?
Same GA settings?
Same curvature calculation?
Same objective weights?
Same obstacle layout?
```

This is a reproducibility task.

---

5. There is one known incompleteness in your Paper 1 implementation

Your documentation itself says:

> F_{dyn} *is omitted.*

Paper 1's objective contains:

$$J = w_1 F_{data} + w_2 F_{curv} + w_3 F_{dyn} + w_4 F_{len} + w_5 F_{knot}$$

while the current implementation uses a reduced four-term objective. `Pasted markdown.md` `Pasted markdown.md`

For now:

Do NOT add F_{dyn}.

Because you're trying to reproduce the current paper implementation first.

Just document:

> *"Dynamic feasibility cost term is reserved for the extension phase."*

That's much cleaner.

PHASE 3 - Reproduce Paper 2 independently

Forget the B-Spline for a moment.

Run the quaternion controller against simple references.

The paper tests:

Test 1

Step rotation

Test 2

Sinusoidal reference

Test 3

360° flip

The paper explicitly evaluates these three scenarios. `0602545d-34e1-4c8c-ade0-54f3b41e553b.pdf` `0602545d-34e1-4c8c-ade0-54f3b41e553b.pdf`

Your teammate already has a `paper2_recreation.py`, so use that. `Pasted markdown.md`

Generate:

```
Reference quaternion
Actual quaternion
Quaternion error
```

```
ωx, ωy, ωz
τx, τy, τz
```

---

6. Verify the most important Paper 2 equations

Before moving forward, verify:

Quaternion multiplication

$$p \otimes q$$

Conjugate

$$q^* = [q_0, -q_1, -q_2, -q_3]$$

Error

$$q_e = q_{ref} \otimes q_m^{-1}$$

Control

$$\tau = -P_q q_{e,v} - P_\omega \omega$$

These correspond to the paper's formulation. `0602545d-34e1-4c8c-ade0-54f3b41e553b.pdf` `0602545d-34e1-4c8c-ade0-54f3b41e553b.pdf`

---

7. One thing you MUST verify here

Your documentation writes:

$$\dot{q} = \frac{1}{2} q \otimes \begin{bmatrix} 0 \\ \omega \end{bmatrix}$$

but Paper 2's final quadrotor model is written using the opposite ordering/sign convention:

$$\dot{q} = -\frac{1}{2} \begin{bmatrix} 0 \\ \omega \end{bmatrix} \otimes q$$

`Pasted markdown.md` `0602545d-34e1-4c8c-ade0-54f3b41e553b.pdf`

This can be explained by different frame/convention choices, **but you cannot simply assume they are equivalent**.

So before continuing:

> *Verify the actual quaternion convention used consistently by the code.*

Document:

```
Quaternion order:
[q0 q1 q2 q3]

Angular velocity:
body/world?

Rotation:
body → inertial or inertial → body?

Hamilton product:
left/right convention?

qdot equation:
exact convention?
```

This is a priority-1 check.

PHASE 4 - Validate the existing combination

Only after Paper 1 and Paper 2 independently work:

```
B-Spline
   ↓
C'(u)
   ↓
Tangent
   ↓
q_ref
   ↓
q_error
   ↓
P² controller
```

```
    ↓
τ
    ↓
rotational dynamics
```

Your teammate has implemented exactly this coupling. `Pasted markdown.md`

Now test a VERY simple case first.

---

8. Don't start with a huge obstacle environment

Start with:

```
Start
  \
   \
    \
     Goal
```

Then:

```
simple curved path
```

Then:

```
S-shaped path
```

Then:

```
3D obstacle path
```

This allows you to see where errors begin.

---

9. What should you plot?

At minimum, create one dashboard containing:

Path

$$C(u)$$

Tangent

$$C'(u)$$

Desired quaternion

$$q_{ref}$$

Actual quaternion

$$q$$

Quaternion error

$$q_e$$

Angular velocity

$$\omega$$

Torque

$$\tau$$

Path progress

$$u(t)$$

The current project already generates diagnostic figures, which is a good foundation.
`Pasted markdown.md`

---

10. What does "success" mean for this stage?

You should be able to demonstrate:

```
B-Spline path
      ↓
tangent changes
      ↓
q_ref changes
      ↓
P² controller tracks q_ref
```

```
      ↓
attitude error → small
```

At this point you can confidently say:

> *"We have successfully integrated the path-smoothing framework with the quaternion attitude-control framework."*

That is your first milestone.

PHASE 5 - Verify the current A* + B-Spline + controller pipeline

Now introduce the actual 3D environment.

The documentation says the system currently contains:

- 3D A*
- 26-connected grid
- cylindrical/cuboidal obstacles
- safety buffers
- B-Spline smoothing
- attitude controller. `Pasted markdown.md` `Pasted markdown.md`

Run:

```
3D Environment
      ↓
A*
      ↓
Raw waypoints
      ↓
Adaptive B-Spline
      ↓
q_ref
      ↓
P² controller
```

But still **don't add position control yet**.

Your goal here is simply to validate the architecture your teammate already created.

---

11. Your milestone checklist

I would use this exact checklist.

Milestone 1 - B-Spline

- [ ] Cubic B-Spline works
- [ ] Cox-de Boor verified
- [ ] Control points verified
- [ ] Knot vector verified
- [ ] Curvature verified
- [ ] Path length verified

Milestone 2 - GA

- [ ] GA modifies only valid interior knots
- [ ] Knot vector stays sorted
- [ ] Fitness calculation verified
- [ ] Best solution reproducible with seed
- [ ] Compare against uniform/chord-length

Milestone 3 - Curvature refinement

- [ ] Maximum curvature identified
- [ ] Correct local control points identified
- [ ] Refinement lowers objective
- [ ] No collision introduced

Milestone 4 - Quaternion

- [ ] Hamilton product verified
- [ ] Conjugate verified

- [ ] Inverse verified
- [ ] Quaternion norm stays near 1
- [ ] q-error verified
- [ ] frame convention verified

Milestone 5 - P² controller

- [ ] Step tracking
- [ ] Sinusoidal tracking
- [ ] Flip
- [ ] Torque saturation
- [ ] noise test

Milestone 6 - Integration

- [ ] tangent → q_ref
- [ ] q_ref continuity
- [ ] SLERP
- [ ] q_error
- [ ] P² controller
- [ ] path progression

Milestone 7 - Complete baseline

- [ ] 3D A*
- [ ] B-Spline
- [ ] GA
- [ ] curvature refinement
- [ ] quaternion controller
- [ ] combined simulation
- [ ] final plots

Only after Milestone 7 do we start your novel work.

12. What about MATLAB vs Python?

For this first stage, **don't maintain two independent implementations unless necessary**.

Your teammate has both Python and MATLAB versions. `Pasted markdown.md`

I would do:

Python

Use it for:

```
Algorithm development
GA
B-Spline
experiments
parameter sweeps
plots
```

MATLAB/Simulink

Use it for:

```
control validation
dynamic simulation
visualization
later 6-DOF model
```

For now, pick **Python as the baseline implementation you actively develop**, while using MATLAB as a validation/reference environment.

Otherwise you will spend a lot of time maintaining two versions of every change.

---

13. What you should NOT do yet

Don't start:

❌ MPC

❌ RL

❌ Dynamic obstacles

❌ wind model

❌ neural networks

❌ adaptive GA

❌ jerk optimization

❌ real drone

❌ PX4

❌ complicated 6-DOF model

Not yet.

All of those are excellent **later**.

Right now they will make it impossible to determine whether the basic Paper 1 + Paper 2 integration actually works.

---

14. Your project roadmap

I would structure the whole project like this:

```
PHASE 1
Reproduce Paper 1
        ↓
PHASE 2
Reproduce Paper 2
        ↓
PHASE 3
Combine Paper 1 + Paper 2
        ↓
PHASE 4
Validate complete baseline
        ↓
═══════════════════════════
       BASELINE COMPLETE
═══════════════════════════
        ↓
PHASE 5
Extended work
        ↓
PHASE 6
Comparative experiments
```

```
        ↓
PHASE 7
Research paper
```

This is a very clean research story.

---

15. And your future extension should come AFTER this

Once the baseline is stable, your strongest next move would be:

```
Current:
B-Spline → tangent → quaternion → P²

              ↓

Extended:
B-Spline → position/velocity/acceleration
              ↓
       position control
              ↓
      desired acceleration
              ↓
      desired attitude
              ↓
      quaternion P² control
              ↓
        6-DOF dynamics
```

Then later:

```
κ(u)
 ↓
adaptive velocity
```

and:

```
dynamic obstacles
 ↓
replanning
```

and:

```
wind/noise
 ↓
robustness testing
```

That gives you a natural progression from **reproduction → integration → extension → research contribution**.

---

16. So, was your teammate's partial work correct?

My assessment based on the documentation is:

Yes, the direction is correct.

The existing combination:

$$\boxed{\text{Adaptive B-Spline} \rightarrow \text{Path Tangent} \rightarrow q_{ref} \rightarrow \text{Quaternion P}^2}$$

is a legitimate first integration. `Pasted markdown.md`

But call it:

> ***baseline combined path-smoothing and attitude-control framework***

-not a complete UAV trajectory-control system yet.

The two things I would verify **before doing anything else** are:

1. **Paper 1 reproducibility**, because the reported numbers currently don't match the original paper exactly.
2. **Quaternion convention consistency**, because the documented kinematic equation differs in ordering/sign from the paper's final model and needs to be reconciled explicitly.

After those are settled, your next actual coding task is very simple:

> ***Run Paper 1 alone → run Paper 2 alone → run the existing combined tangent-to-quaternion simulation → produce the three sets of results.***

That gives you a clean, defensible baseline before we touch any extension. `Pasted markdown.md`

8/15/2026, 8:08:11 PM